Git & GitHub with VS Code

MODULE 1: Setup & Fundamentals

1.1 Environment Setup

Install Required Software

1. **Git** - Download from https://git-scm.com/
   - Windows: Run installer with default settings
   - Mac: Install via terminal or download
   - Verify: Open terminal and run `git --version`

2. **VS Code** - Download from https://code.visualstudio.com/
   - Install with default settings

3. **Create GitHub Account** - https://github.com

VS Code Extensions to Install

Open VS Code → Extensions (Ctrl+Shift+X / Cmd+Shift+X)

Essential Extensions

GitLens (by GitKraken) - Supercharges Git capabilities
Git Graph (by mhutchie) - Visualize branch history
GitHub Pull Requests and Issues (by GitHub) - Manage PRs in VS Code
Git History (by Don Jayamanne) - View file/line history

Optional but Recommended

Prettier - Code formatter
Live Share - Real-time collaboration

Configure Git Identity

Open VS Code Terminal (Ctrl+` or View → Terminal):

```bash
git config --global user.name "Your Full Name"
git config --global user.email "youremail@example.com"
git config --global init.defaultBranch main
```

Verify configuration:

```
git config --list
```

Git Workflow Overview

```
Working Directory → Staging Area → Local Repository → Remote Repository
     (edit)           (git add)      (git commit)      (git push)
```

1.2 First Git Repository

Step 1: Create Project Folder

1. Create folder: data-pipeline-project
2. Open in VS Code: File → Open Folder
3. Create file: etl_pipeline.py

Step 2: Initialize Git Repository

- Open VS Code terminal

```
git init
```

- Notice: Source Control icon shows changes

Step 3: Understanding VS Code Source Control Panel

- Click Source Control icon (left sidebar)
- See "Changes" section with etl_pipeline.py
- This is your "Working Directory"

Step 4: Stage Your Changes Two ways:

- **Terminal:** git add etl_pipeline.py or git add .

Step 5: Commit Your Changes

- **Terminal:** git commit -m "Initial commit: Basic ETL pipeline"
- **IMPORTANT: Create .gitignore** (before committing!)
- **Branching:** git branch -M main
- **Send to github:** git remote add origin "the link to the repo"
- **Push:** git push -u origin main

Step 6: View Your History

- Git status

Module 2: Cloning, Forking & GitHub Integration (2 hours)

2.1 Understanding Clone vs Fork (20 minutes)

Theory: Clone vs Fork - What's the Difference?

CLONING:

- Creates a local copy of a repository on your computer
- Direct connection to the original repository
- You need write access to push changes directly
- Best for: Your own repos, team projects where you're a collaborator

FORKING:

- Creates your own copy of someone else's repository on GitHub
- Your copy is independent but connected to the original
- You can freely experiment without affecting the original
- Submit changes via Pull Requests
- Best for: Contributing to open-source, learning from others' code

Visual Comparison:

```
CLONE:
GitHub Repo → (download) → Your Computer
              git clone
              ↕ (can push if you have access)

FORK:
Original GitHub Repo → (copy on GitHub) → Your GitHub Fork → (download) → Your
Computer
                       click Fork          git clone          ↕ (you own this)
```

2.2 Cloning Repositories (30 minutes)

HANDS-ON EXERCISE 3: Clone a Repository

Scenario 1: Clone Your Own Repository

Step 1: Get the Clone URL

- Click green "Code" button
- Copy the HTTPS URL:
  https://github.com/yourusername/python-data-analysis.git

Step 2: Clone via VS Code

On Terminal:

```
# Navigate to where you want the project
cd ~/Documents/projects

# Clone the repository
git clone https://github.com/yourusername/python-data-analysis.git

# Navigate into it
cd python-data-analysis

# Open in VS Code
code .
```

Step 4: Verify the Clone

```
# Check remote connection
git remote -v

# You should see:
# origin  https://github.com/yourusername/python-data-analysis.git (fetch)
# origin  https://github.com/yourusername/python-data-analysis.git (push)
```

Forking Repositories

Fork and Contribute

Scenario: You want to contribute to an open-source data engineering project

Step 1: Fork a Repository on GitHub

For Practice - Fork a Sample Repo:

1. Go to: https://github.com/yourinstructor/sample-data-project (or any public repo)
2. Click "Fork" button (top-right)
3. GitHub creates a copy under your account: https://github.com/yourusername/sample-data-project

Understanding what happened:

- Original repo: yourinstructor/sample-data-project (upstream)
- Your fork: yourusername/sample-data-project (origin)
- You now own your fork and can do anything with it!

Step 2: Clone YOUR Fork

```
cd ~/Documents/projects
git clone https://github.com/yourusername/sample-data-project.git
cd sample-data-project
```

Step 3: Set Up Upstream Remote This keeps your fork connected to the original repository:

```
# Add the original repository as 'upstream'
git remote add upstream
https://github.com/yourinstructor/sample-data-project.git

# Verify remotes
git remote -v

# You should see:
# origin    https://github.com/yourusername/sample-data-project.git (fetch)
# origin    https://github.com/yourusername/sample-data-project.git (push)
# upstream  https://github.com/yourinstructor/sample-data-project.git (fetch)
# upstream  https://github.com/yourinstructor/sample-data-project.git (push)
```

Understanding Remotes:

- **origin:** Your fork (you can push here)
- **upstream:** Original repo (you pull updates from here)

How to Make Changes to Forked Repo

Step 1: Create a feature branch

```
git checkout -b feature/add-data-cleaning
```

Step 2: Add your contribution

In a file write the code you want to add to the upstream repo.

Step 3: Commit your changes

```
git add name_used_to_your_file
git commit -m "Add your commit message here”
```

Step 4: Push to YOUR fork

```
git push -u origin feature/add-data-cleaning
```

Step 5: Create Pull Request on GitHub with proper documentation

Complete Workflow:

```
# 1. Always start by syncing
git checkout main
git fetch upstream
git merge upstream/main
git push origin main

# 2. Create feature branch FROM updated main
git checkout -b feature/new-contribution

# 3. Make changes, commit
git add .
git commit -m "Add new feature"

# 4. Push feature branch
git push origin feature/new-contribution

# 5. Create PR on GitHub
```

Merging Branches

Merge via Pull Request

Step 1: Create Pull Request on GitHub

1. Go to your repo on GitHub
2. Click "Compare & pull request" (appears after pushing branch)
3. Add description: "Adds database connector module with support for PostgreSQL, MySQL, and SQLite"
4. Click "Create pull request"

Step 2: Review and Merge

1. Review the changes (Files changed tab)
2. Click "Merge pull request"
3. Click "Confirm merge"
4. Delete the branch (optional but recommended)

Step 3: Update local repository

```
git checkout main
git pull
```

Your main branch now has the database connector feature!

4. **Merge locally:**

```
git checkout main
git merge feature/data-transformations
```

5. Push to GitHub:

```
git push
```

Resources & Cheat Sheets

Essential Git Commands for Data Engineering

```
# Setup
git config --global user.name "Name"
git config --global user.email "email@example.com"
git config --global init.defaultBranch main

# Repository Management
git init
git clone <url>
git remote add upstream <url>  # For forks

# Daily Workflow
git status
git add <file>
git add .
git commit -m "message"
git push
git pull
git fetch

# Branching
git branch                    # List branches
git branch <name>             # Create branch
git checkout <name>           # Switch branch
git checkout -b <name>        # Create and switch
git merge <branch>            # Merge branch
git branch -d <name>          # Delete branch

# Syncing Fork
git fetch upstream
git merge upstream/main
git push origin main

# Stashing
git stash
git stash pop
git stash list
git stash apply

# History
git log
git log --oneline
git log --graph --oneline --all

# Undoing
git checkout -- <file>         # Discard changes
git reset HEAD <file>          # Unstage
git reset --soft HEAD~1        # Undo commit, keep changes
git reset --hard HEAD~1        # Undo commit, discard changes
git revert <commit>            # Safe undo for pushed commits

# Advanced
git cherry-pick <commit>       # Apply specific commit
git rebase <branch>            # Rebase current branch
git rebase -i HEAD~3          # Interactive rebase
```

```
# Information
git remote -v                  # Show remotes
git diff                       # Show changes
git diff --staged              # Show staged changes
git show <commit>              # Show commit details
```

Data Engineering-Specific Best Practices

1. **Never commit:**
   - Large data files (use .gitignore or Git LFS)
   - Credentials (.env files)
   - Database dumps
   - Jupyter notebook outputs
   - API keys
2. **Always commit:**
   - Source code
   - Requirements files
   - Configuration templates (.env.example)
   - Documentation
   - Tests
   - Schema definitions
3. **Commit Message Format:**
   - feat(module): add feature description
   - Detailed explanation of what changed and why.
     - Bullet point 1
     - Bullet point 2
     Closes #123
4. **Branch Naming:**
   - feature/add-mysql-connector
   - bugfix/fix-null-handling
   - hotfix/critical-data-loss
   - refactor/optimize-queries